KIET College ERP Portal

Software Metrics & Complexity Analysis

Academic Project Report

1. Introduction

Software metrics are measurable indicators used to evaluate software size, complexity, quality, maintainability, and performance. Complexity analysis estimates how much processing time and memory an algorithm or software operation may require. This report applies these ideas to the proposed KIET College ERP Portal described in the project SRS.

Scope note: The SRS identifies ERP modules and requirements but does not provide source code, measured execution times, function-point counts, or measured complexity values. Therefore, the metrics below are qualitative or illustrative estimates for an academic design report, not results measured from an implemented system.

2. Software Metrics

Software metrics help the team assess whether the ERP portal is understandable, testable, reliable, and maintainable. The following metrics are suitable for this project.

Metric	Meaning	Application to KIET ERP Portal	How to Measure
Size	Amount of software built	Number of modules, screens, requirements, or lines of code	Count after implementation; LOC is not available at SRS stage
Cyclomatic Complexity	Number of independent paths through a function	Can be applied to login validation, attendance validation, marks entry, and result publication logic	V(G) = E – N + 2P for a control-flow graph; commonly approximated as decisions + 1 for one connected function
Function Points (FP)	Functional size based on user-visible inputs, outputs, inquiries, files, and interfaces	Can estimate size of login, attendance, marks, fees, notices, reports, and timetable functions	Count and weight function types using an agreed FP method; detailed counts are not present in SRS
Defect Density	Defects relative to software size	Track defects found in modules such as attendance, fee status, and results	Defect density = confirmed defects / size unit (e.g., KLOC or FP)
Test Coverage	How much code or requirements are tested	Map test cases to FR-01 to FR-18 and relevant security requirements	Coverage = tested items / total applicable items × 100%
Maintainability	Ease of changing and fixing software	Separate authentication, academics, examination, fees, notices, and reporting into modules	Review coupling, cohesion, change effort, documentation, and code complexity
Reliability / Availability	Consistency of correct operation and access over time	Important during attendance entry, exams, and result publication	Measure failure rate, recovery time, uptime, and incident records after deployment

2.1 Example Metrics for Design Review

At the SRS stage, the team can track requirement coverage and module completeness without claiming implementation measurements. The SRS lists 18 functional requirements (FR-01 to FR-18), and a first-pass traceability check can count how many have at least one planned test case.

Design-stage measure	Example / interpretation
Functional requirements	18 listed requirements (FR-01 to FR-18) in the SRS

Design-stage measure	Example / interpretation
Planned requirement test coverage	If test cases exist for 12 of 18 requirements, planned coverage = 12/18 × 100 = 66.7%. This is an example only, not the project's measured coverage.
Module coverage	Check that authentication, student/faculty, academics, attendance, timetable, exams/results, fees, notices, and reports each have requirements and test cases.

3. Complexity Analysis

Complexity analysis studies how the resource needs of an operation grow as the input size increases. Time complexity describes growth in processing steps; space complexity describes extra memory required. Big-O notation is commonly used to express an upper-bound growth rate.

3.1 Common Complexity Classes

Notation	Name	Simple interpretation	ERP-related example (illustrative)
O(1)	Constant	Work stays roughly constant as records increase	Accessing a record by a unique indexed ID, assuming an effective index
O(log n)	Logarithmic	Work grows slowly as input grows	Searching a sorted in-memory list using binary search
O(n)	Linear	Work grows proportionally to number of records	Scanning a list of students to find a matching record
O(n log n)	Linearithmic	Typical growth for efficient comparison sorting	Sorting a large list of report rows with a comparison sort
$O(n^2)$	Quadratic	Nested comparisons can grow rapidly	Comparing every student record with every other record in a naive approach
O(1) auxiliary space	Constant extra space	Extra memory does not grow with input size	Processing records one at a time without storing a second full copy
O(n) auxiliary space	Linear extra space	Extra memory grows with number of records	Loading all report rows into memory before processing

3.2 Complexity of Key ERP Operations

The SRS defines the workflows but does not prescribe algorithms or database indexes. The following are reasonable design-level expectations only; actual complexity depends on implementation, query plans, indexes, and data structures.

Operation	Expected complexity / cost	Notes
Login and role lookup	Often approximately O(1) average lookup with a suitable indexed unique identifier; password verification has its own fixed cost per configured algorithm	Depends on database indexing and authentication design.
View attendance	Database query cost depends on indexes and number of matching records; returning k records requires at least O(k) output work	Index student ID, subject, and/or date where appropriate.
Mark attendance for n students	O(n) record processing for n submitted attendance entries	Validation and database writes are performed per entry; transaction strategy affects practical cost.
Enter marks for n students	O(n) validation/processing for n marks	Validate allowed mark ranges and authorization before storing.
Generate a report for n matching rows	At least O(n) to process/output all rows; sorting may add O(n log n)	Database filtering and aggregation can change the total query cost.
View fee status	Often efficient indexed lookup for one student; result size and query plan matter	Use authorized access to protect financial records.

4. Improving Efficiency and Maintainability

- Use indexes for frequently searched identifiers such as student ID, subject, exam, and fee record keys where appropriate.
- Validate inputs at the application boundary and enforce authorization for protected records.
- Use pagination for large attendance, marks, and report tables rather than loading every row at once.
- Keep modules separate and use shared validation/authentication logic to reduce duplication.
- Measure response time and database query performance with realistic test data before making performance claims.
- Test important workflows such as login, attendance entry, marks entry, result publication, and fee administration.

5. Conclusion

Software metrics provide a structured way to evaluate the ERP portal's size, quality, testing progress, and maintainability. Complexity analysis helps estimate how attendance, marks, reporting, and other operations may scale as records grow. Since the supplied SRS is a requirements document rather than an implemented codebase, numeric runtime and code-complexity results should be measured after implementation.

Reference basis: KIET College ERP Portal SRS, Version 1.0 — functional requirements FR-01 to FR-18, major product functions, security requirements, and quality attributes.